

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_c5395511-084\agent-a0f3aeb1d20e6ab.jsonl`
- SHA-256: `9dce271f40c37adfd62a5ede110d7b31b878f43298e3ee82ac54980a17ecee43`
- Source modified: `2026-07-06T21:29:35+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf\_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:25:50.740Z)

Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there, or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (db._apply_schema on aiosqlite ":memory:").

WHAT F2 DOES:

- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video / _content_item_to_video / get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences (json.loads(policy.config_json)); TopicPreferenceStage (Tier 2): exclude-veto, include any/all, casefold substring match over source.title + content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty); _content_item_preview(...) builds a throwaway ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn, account, source, item) loads the default policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now: matched=engine.evaluate(); if matched and source: run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a /set (F1's merge) and are read via the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate: ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium (edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

DIMENSION: design consistency, caption-capture completeness, and test coverage.

Investigate:

- evaluate_pipeline BYPASS: F1 added evaluate_pipeline as the orchestrator, but F2 hand-wires TopicPreferenceStage directly (FilterEngine is not a PolicyStage). So evaluate_pipeline stays unused in production and the unified StageResult

explanation trace isn't built. Design-consistency issue?

- No user-facing way to set 'topics' in F2 (no /set topics command) -> the whole stage is dormant until a config path exists. Scope note; disclosed?
- Caption capture completeness: is VideoRecord.text propagated in ALL shims (insert_video, _content_item_to_video, get_video_by_content_fingerprint, list_videos_for_user, get_video/get_video_by_document_id)? Any read path that drops text? Does backfill store text too?
- Test coverage: TopicPreferenceStage is unit-tested; but the WIRING (_evaluate_topic_preferences, _content_item_preview, the on_new_message/backfill gating) is behind pragma:no-cover and appears untested. coverage dropped 96.63->95.58. Which new lines are uncovered? A test would catch a wiring bug.
- _content_item_preview computes a dedup_key that is never used (throwaway) ? wasteful? any correctness impact?

Run repros. Report findings.

2. assistant (2026-07-06T21:25:53.332Z)

I'll review PR #51 F2 topic preferences. Let me start by reading the changed files.

3. user (2026-07-06T21:25:56.220Z)

```
src/tg_video_filter/backfill.py      | 52 ++++++++--
src/tg_video_filter/db.py           |  4 ++
src/tg_video_filter/models.py        |  1 +
src/tg_video_filter/policy.py        | 117 ++++++++
src/tg_video_filter/telethon_client.py | 115 ++++++++
tests/test_db.py                    |  8 +++
tests/test_policy.py                 | 110 ++++++++
tests/test_telethon_extraction.py    | 26 +++++-
8 files changed, 417 insertions(+), 16 deletions(-)
```

4. user (2026-07-06T21:25:56.850Z)

tg_video_filter

5. assistant (2026-07-06T21:25:58.875Z)

Let me read the key files. I'll read the diffs and full new code.

6. user (2026-07-06T21:26:01.433Z)

```
diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
new file mode 100644
index 0000000..395435a
--- /dev/null
+++ b/src/tg_video_filter/policy.py
@@ -0,0 +1,199 @@
+"""Policy pipeline primitives for ADR 0012.
+
++The policy layer is deliberately small and framework-free in F1: stages receive
++an immutable context, return a structured result, and the evaluator stops on
++the first failure. Later Track F phases add concrete stages.
+"""
+
++from __future__ import annotations
++
++import json
++from dataclasses import dataclass, field
++from typing import Protocol
++
++from tg_video_filter.models import ContentItem, Policy, Source
++
++
++@dataclass(frozen=True)
```

```

+class PolicyContext:
+    """Immutable snapshot available to a policy stage."""
+
+    content_item: ContentItem
+    source: Source
+    policy: Policy
+    fetcher: object | None = None
+    budget: object | None = None
+
+
+@dataclass(frozen=True)
+class StageResult:
+    """Outcome of one policy stage."""
+
+    passed: bool
+    reason: str
+    stage_name: str
+    metadata: dict[str, object] = field(default_factory=dict)
+
+    @staticmethod
+    def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
+        return StageResult(
+            passed=True,
+            reason=reason,
+            stage_name=stage_name,
+            metadata=metadata,
+        )
+
+    @staticmethod
+    def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
+        return StageResult(
+            passed=False,
+            reason=reason,
+            stage_name=stage_name,
+            metadata=metadata,
+        )
+
+
+class PolicyStage(Protocol):
+    """A single stateless gate in the policy pipeline."""
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        """Evaluate ``ctx`` and return a stage result."""
+        ...
+
+
+@dataclass(frozen=True)
+class TopicPreferences:
+    """Parsed ADR 0012 topic preference config."""
+
+    include: tuple[str, ...] = ()
+    exclude: tuple[str, ...] = ()
+    mode: str = "any"
+
+    @property
+    def enabled(self) -> bool:
+        return bool(self.include or self.exclude)
+
+
+def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
+    """Parse ``topics`` from a raw policy config object."""
+    raw_topics = config.get("topics")
+    if raw_topics is None:
+        return TopicPreferences()
+    if not isinstance(raw_topics, dict):

```

```

+         raise ValueError("topics must be a JSON object")
+     mode = raw_topics.get("mode", "any")
+     if mode not in ("any", "all"):
+         raise ValueError("topics.mode must be 'any' or 'all'")
+     return TopicPreferences(
+         include=_string_tuple(raw_topics.get("include")),
+         exclude=_string_tuple(raw_topics.get("exclude")),
+         mode=str(mode),
+     )
+
+
+def load_topic_preferences(policy: Policy) -> TopicPreferences:
+    """Load topic preferences from ``policy.config_json``."""
+    decoded = json.loads(policy.config_json)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return parse_topic_preferences(decoded)
+
+
+def _string_tuple(value: object) -> tuple[str, ...]:
+    if value is None:
+        return ()
+    if not isinstance(value, list):
+        raise ValueError("topic include/exclude values must be arrays")
+    topics: list[str] = []
+    for item in value:
+        if not isinstance(item, str):
+            raise ValueError("topic include/exclude arrays must contain strings")
+        topic = item.strip()
+        if topic:
+            topics.append(topic)
+    return tuple(topics)
+
+
+class TopicPreferenceStage:
+    """ADR 0012 Tier 2 topic matching over source title and captured caption."""
+
+    stage_name = "topic-preferences"
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        try:
+            prefs = load_topic_preferences(ctx.policy)
+            except (json.JSONDecodeError, ValueError) as exc:
+                return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
+            if not prefs.enabled:
+                return StageResult.pass_(self.stage_name, "topic preferences disabled")
+
+            haystack = _topic_haystack(ctx)
+            matched_excludes = _matched_topics(prefs.exclude, haystack)
+            if matched_excludes:
+                return StageResult.fail(
+                    self.stage_name,
+                    f"excluded topic matched: {matched_excludes[0]}",
+                    matched_exclude_topics=list(matched_excludes),
+                )
+
+            if not prefs.include:
+                return StageResult.pass_(self.stage_name, "no include topics configured")
+
+            matched_includes = _matched_topics(prefs.include, haystack)
+            if prefs.mode == "any":
+                if matched_includes:
+                    return StageResult.pass_(
+                        self.stage_name,
+                        f"matched include topic: {matched_includes[0]}",

```

```

+         matched_include_topics=list(matched_includes),
+     )
+     return StageResult.fail(self.stage_name, "no include topics matched")
+
+     missing = tuple(topic for topic in prefs.include if topic not in matched_includes)
+     if missing:
+         return StageResult.fail(
+             self.stage_name,
+             f"missing required topic: {missing[0]}",
+             matched_include_topics=list(matched_includes),
+             missing_include_topics=list(missing),
+         )
+     return StageResult.pass_(
+         self.stage_name,
+         "matched all include topics",
+         matched_include_topics=list(matched_includes),
+     )
+
+
+def _topic_haystack(ctx: PolicyContext) -> str:
+    parts = [ctx.source.title or "", ctx.content_item.text or ""]
+    return "\n".join(parts).casefold()
+
+
+def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
+    return tuple(topic for topic in topics if topic.casefold() in haystack)
+
+
+async def evaluate_pipeline(
+    ctx: PolicyContext,
+    stages: list[PolicyStage],
+) -> tuple[bool, list[StageResult]]:
+    """Evaluate stages in order and short-circuit on the first failure."""
+    results: list[StageResult] = []
+    for stage in stages:
+        result = await stage.evaluate(ctx)
+        results.append(result)
+        if not result.passed:
+            return False, results
+    return True, results
+
+
+__all__ = [
+    "PolicyContext",
+    "PolicyStage",
+    "StageResult",
+    "TopicPreferenceStage",
+    "TopicPreferences",
+    "evaluate_pipeline",
+    "load_topic_preferences",
+    "parse_topic_preferences",
+]

```

7. user (2026-07-06T21:26:02.135Z)

```

diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index df8b4f9..7b393f8 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -35,7 +35,16 @@ from tg_video_filter import db
     from tg_video_filter.commands import ENGINE_KEY
     from tg_video_filter.delivery import build_link_text, deliver
     from tg_video_filter.filters import FilterEngine, VideoMeta
-    from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord
+    from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord
+
+

```

```

+ Channel,
+ ContentItem,
+ DeliveryMode,
+ Source,
+ TelegramSession,
+ User,
+ VideoRecord,
+)
+from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage

log = logging.getLogger(__name__)

@@ -243,29 +252,56 @@ def make_new_message_handler(
    )
    return

+
+    source = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
+
+    # Lazily record the channel (best-effort; failures are non-fatal).
+    try:
+        await db.upsert_channel(
+        channel = await db.upsert_channel(
+            conn,
+            Channel(user_id=user.id, channel_id=channel_id, title=_channel_title(event)),
+        )
+        if channel.id is not None:
+            source = await db.get_source_by_id(conn, channel.id)
+    except Exception:
+        log.warning(
+            "failed to upsert channel %s for user %s", channel_id, user.id, exc_info=True
+        )

+
+    text = _message_text(message)
+    result = engine_holder[ENGINE_KEY].evaluate(meta)
+    matched = result.matched
+    reasons = list(result.reasons)
+    if matched and source is not None:
+        topic_result = await _evaluate_topic_preferences(
+            conn,
+            user.id,
+            source,
+            _content_item_preview(
+                user.id,
+                source,
+                message_id,
+                document_id,
+                meta,
+                text,
+                matched,
+            ),
+        )
+        if topic_result is not None:
+            reasons.append(topic_result.reason)
+            matched = topic_result.passed
+
+    record = VideoRecord(
+        user_id=user.id,
+        channel_id=channel_id,
+        message_id=message_id,
+        text=text,
+        document_id=document_id,
+        duration_s=meta.duration_s,
+        width=meta.width,
+        height=meta.height,
+        size_bytes=meta.size_bytes,

```

```

        mime_type=meta.mime_type,
-        matched=result.matched,
+        matched=matched,
    )
    try:
        inserted = await db.insert_video(conn, record)
@@ -288,11 +324,11 @@ def make_new_message_handler(
    user.id,
    channel_id,
    message_id,
-    result.matched,
-    result.reasons,
+    matched,
+    reasons,
)

-    if result.matched:
+    if matched:
        # Build link text for LINK/BOTH modes (None for FORWARD).
        link_text = None
        if delivery_mode != DeliveryMode.FORWARD:
@@ -335,6 +371,70 @@ def _extract_document_id(message: Message) -> int | None:
    return None

+def _message_text(message: object) -> str | None:
+    """Return caption/message text normalized for storage, or None if empty."""
+    text = getattr(message, "message", None)
+    if text is None:
+        text = getattr(message, "raw_text", None)
+    if not isinstance(text, str):
+        return None
+    text = text.strip()
+    return text or None
+
+def _content_item_preview(
+    account_id: int,
+    source: Source,
+    message_id: int,
+    document_id: int | None,
+    meta: VideoMeta,
+    text: str | None,
+    matched: bool,
+) -> ContentItem:
+    """Build the ContentItem shape used by policy stages before insert."""
+    assert source.id is not None
+    return ContentItem(
+        account_id=account_id,
+        source_id=source.id,
+        provider=source.provider,
+        provider_message_id=str(message_id),
+        content_type="video",
+        text=text,
+        document_id=str(document_id) if document_id is not None else None,
+        duration_s=meta.duration_s,
+        width=meta.width,
+        height=meta.height,
+        size_bytes=meta.size_bytes,
+        mime_type=meta.mime_type,
+        dedup_key=db.compute_dedup_key(
+            provider=source.provider,
+            document_id=str(document_id) if document_id is not None else None,
+            source_id=source.id,
+            provider_message_id=str(message_id),

```

```

+         size_bytes=meta.size_bytes,
+         duration_s=meta.duration_s,
+         height=meta.height,
+         width=meta.width,
+     ),
+     matched=matched,
+ )
+
+
+async def _evaluate_topic_preferences(
+    conn: aiosqlite.Connection,
+    account_id: int,
+    source: Source,
+    content_item: ContentItem,
+) -> StageResult | None:
+    """Evaluate ADR 0012 topic preferences for a candidate item."""
+    policy = await db.get_default_policy(conn, account_id)
+    if policy is None:
+        return None
+    return await TopicPreferenceStage().evaluate(
+        PolicyContext(content_item=content_item, source=source, policy=policy)
+    )
+
+
+def _peer_id(peer: object) -> int:
+    """Best-effort numeric id for the peer of a message.
+
+@@ -367,4 +467,7 @@ __all__ = [
+    "build_client",
+    "extract_video_meta",
+    "make_new_message_handler",
+    "_content_item_preview",
+    "_evaluate_topic_preferences",
+    "_message_text",
+]

```

8. user (2026-07-06T21:26:07.605Z)

```

diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
index 945132b..7a370f2 100644
--- a/src/tg_video_filter/backfill.py
+++ b/src/tg_video_filter/backfill.py
@@ -31,8 +31,15 @@ from tg_video_filter import db
 from tg_video_filter.commands import ENGINE_KEY
 from tg_video_filter.delivery import build_link_text, deliver
 from tg_video_filter.filters import FilterEngine
-from tg_video_filter.models import DeliveryMode, User, VideoRecord
-from tg_video_filter.telethon_client import _extract_document_id, _peer_id, extract_video_meta
+from tg_video_filter.models import Channel, DeliveryMode, User, VideoRecord
+from tg_video_filter.telethon_client import (
+    _content_item_preview,
+    _evaluate_topic_preferences,
+    _extract_document_id,
+    _message_text,
+    _peer_id,
+    extract_video_meta,
+)

log = logging.getLogger(__name__)

@@ -216,22 +223,59 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
    summary["videos_scanned"] += 1
    dialog_videos += 1

```

```

+         source = None
+     try:
+         channel = await db.upsert_channel(
+             conn,
+             Channel(user_id=user.id, channel_id=channel_id, title=title),
+         )
+         if channel.id is not None:
+             source = await db.get_source_by_id(conn, channel.id)
+     except Exception:
+         log.warning(
+             "failed to upsert channel %s for user %s during backfill",
+             channel_id,
+             user.id,
+             exc_info=True,
+         )
+
+     text = _message_text(message)
+     result = engine.evaluate(meta)
+     matched = result.matched
+     if matched and source is not None:
+         topic_result = await _evaluate_topic_preferences(
+             conn,
+             user.id,
+             source,
+             _content_item_preview(
+                 user.id,
+                 source,
+                 message_id,
+                 document_id, # type: ignore[arg-type]
+                 meta,
+                 text,
+                 matched,
+             ),
+         )
+         if topic_result is not None:
+             matched = topic_result.passed
+
+     record = VideoRecord(
+         user_id=user.id,
+         channel_id=channel_id,
+         message_id=message_id,
+         text=text,
+         document_id=document_id, # type: ignore[arg-type]
+         duration_s=meta.duration_s,
+         width=meta.width,
+         height=meta.height,
+         size_bytes=meta.size_bytes,
+         mime_type=meta.mime_type,
+         matched=result.matched,
+         matched=matched,
+     )
+     inserted = await db.insert_video(conn, record)
+
+     if result.matched:
+     if matched:
+         summary["videos_matched"] += 1
+         dialog_matched += 1
+         link_text = build_link_text(
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index a059e57..e6fca83 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -11,6 +11,8 @@ in-memory database (`aiosqlite.connect(":memory:")`).

from __future__ import annotations

```

```

+import json
+import logging
+    from collections.abc import Awaitable, Callable, Iterable
+    from datetime import UTC, datetime

@@ -32,6 +34,8 @@ from tg_video_filter.models import (
    VideoRecord,
)

+log = logging.getLogger(__name__)
+
+    # --- DDL -----
+
+_SCHEMA = """
@@ -1288,6 +1292,56 @@ async def get_default_policy(conn: aiomysqlite.Connection, account_id: int)
-> Pol
    return _row_to_policy(row) if row else None

+def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
+    """Decode a policy config JSON object.
+
+    ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
+    expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
+    nested stage config can coexist.
+
+    """
+    decoded = json.loads(raw)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return decoded
+
+def _policy_config_dict_to_json(config: dict[str, object]) -> str:
+    """Encode policy config deterministically and compactly."""
+    return json.dumps(config, separators=(",", ":"), sort_keys=True)
+
+async def load_policy_config(conn: aiomysqlite.Connection, account_id: int) -> dict[str, object]:
+    """Load the raw default policy config as a JSON object.
+
+    Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
+    as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
+    through the flat ``FilterConfig`` model.
+
+    """
+    policy = await get_default_policy(conn, account_id)
+    if policy is None:
+        return FilterConfig().model_dump(mode="json")
+    return _policy_config_json_to_dict(policy.config_json)
+
+async def save_policy_config(
+    conn: aiomysqlite.Connection, account_id: int, config: dict[str, object]
+) -> None:
+    """Persist the raw default policy config object."""
+    existing = await get_default_policy(conn, account_id)
+    await upsert_policy(
+        conn,
+        Policy(
+            id=existing.id if existing is not None else None,
+            account_id=account_id,
+            name=existing.name if existing is not None else "default",
+            config_json=_policy_config_dict_to_json(config),
+            is_default=True,
+            created_at=(existing.created_at if existing is not None else _now()),
+            updated_at=_now(),

```

```

+         ),
+     )
+
+
# --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --

@@ -1519,6 +1573,7 @@ async def get_video_by_content_fingerprint(
    user_id=user_id,
    channel_id=_safe_int_cast(row["src_pid"], "provider_source_id"),
    message_id=_safe_int_cast(row["provider_message_id"], "provider_message_id"),
+    text=row["text"],
    document_id=(
        _safe_int_cast(row["document_id"], "document_id")
        if row["document_id"] is not None
@@ -1623,6 +1678,7 @@ async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) ->
VideoR
    provider="telegram",
    provider_message_id=str(video.message_id),
    content_type="video",
+    text=video.text,
    document_id=str(video.document_id) if video.document_id is not None else None,
    duration_s=video.duration_s,
    width=video.width,
@@ -1676,6 +1732,7 @@ def _content_item_to_video(
    user_id=user_id,
    channel_id=channel_id,
    message_id=_safe_int_cast(item.provider_message_id, "provider_message_id"),
+    text=item.text,
    document_id=(
        _safe_int_cast(item.document_id, "document_id")
        if item.document_id is not None
@@ -1701,28 +1758,39 @@ async def load_filter_config(conn: aiosqlite.Connection, user_id: int)
-> Filter
    ADR 0011 (Option 2A): ``filter_config`` is renamed to
    ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
    instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
+    Invalid ``config_json`` still raises from JSON/Pydantic validation; callers
+    do not currently treat malformed persisted config as recoverable.
    """
    policy = await get_default_policy(conn, user_id)
    if policy is None:
        return FilterConfig()
-    return FilterConfig.model_validate_json(policy.config_json)
+    return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))

    async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig) ->
None:
-    """Deprecated shim: upsert the account's default :class:`Policy`."""
+    """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
+
+    ADR 0012: future pipeline keys live in the same ``policies.config_json``
+    object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
+    merge the flat config fields into the existing object so unknown policy
+    keys survive legacy ``/set`` command updates.
+    """
    existing = await get_default_policy(conn, user_id)
-    await upsert_policy(
-        conn,
-        Policy(
-            id=existing.id if existing is not None else None,
-            account_id=user_id,
-            name="default",
-            config_json=cfg.model_dump_json(),

```

```

-         is_default=True,
-         created_at=(existing.created_at if existing is not None else _now()),
-         updated_at=_now(),
-     ),
- )
+ raw_config: dict[str, object] = {}
+ if existing is not None:
+     try:
+         raw_config = _policy_config_json_to_dict(existing.config_json)
+     except (json.JSONDecodeError, ValueError):
+         log.warning(
+             "resetting malformed policy config while saving FilterConfig user=%s "
+             "policy_id=%s",
+             user_id,
+             existing.id,
+             exc_info=True,
+         )
+         raw_config = {}
+ raw_config.update(cfg.model_dump(mode="json"))
+ await save_policy_config(conn, user_id, raw_config)

# --- bulk helpers (used by the manager on startup) -----
@@ -1748,6 +1816,7 @@ async def list_videos_for_user( # v2: used by on-demand /search; not
called in
    user_id=user_id,
    channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
    message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
+    text=r["text"],
    document_id=(
        _safe_int_cast(r["document_id"], "document_id")
        if r["document_id"] is not None
@@ -1789,6 +1858,8 @@ __all__ = [
    "get_content_item_by_dedup_key",
    "upsert_policy",
    "get_default_policy",
+    "load_policy_config",
+    "save_policy_config",
    # Destination / Routing / Delivery repos (ADR 0011, Track C)
    "upsert_destination",
    "get_destination",
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index 4988efe..70e2a8a 100644
--- a/src/tg_video_filter/models.py
+++ b/src/tg_video_filter/models.py
@@ -394,6 +394,7 @@ class VideoRecord(BaseModel):
    user_id: int
    channel_id: int = Field(description="Origin Telegram channel id")
    message_id: int = Field(description="Telegram message id within the channel")
+    text: str | None = Field(default=None, description="Caption / message text")
    document_id: int | None = Field(
        default=None,
        description="Telegram document.id ? reused across forwards; Tier-1 content dedup",

```

9. assistant (2026-07-06T21:26:11.565Z)

Now let me look at the read paths that were NOT touched ? `get\_video` and `get\_video\_by\_document\_id` ? to check if they drop text. Also let me look at how the `on\_new\_message` uses `source` before it's first set.

10. user (2026-07-06T21:26:13.334Z)

```

1155:async def get_source_by_provider_id(
1503:async def get_video(
1519:async def get_video_by_document_id(

```

```
1533:async def get_video_by_content_fingerprint(
1593:async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
1720:def _content_item_to_video(
```

11. user (2026-07-06T21:26:16.619Z)

```
1503     async def get_video(
1504         conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
1505     ) -> VideoRecord | None:
1506         """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1507
1508         Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1509         :class:`Source` + :class:`ContentItem` and projects the latter onto a
1510         :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1511         """
1512         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1513         if src is None:
1514             return None
1515         item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1516         return _content_item_to_video(item, src, user_id, channel_id) if item else None
1517
1518
1519     async def get_video_by_document_id(
1520         conn: aiosqlite.Connection, user_id: int, document_id: int
1521     ) -> VideoRecord | None:
1522         """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1523         dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1524         item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1525         if item is None:
1526             return None
1527         src = await get_source_by_id(conn, item.source_id)
1528         if src is None:
1529             return None
1530         return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1531
1532
1533     async def get_video_by_content_fingerprint(
1534         conn: aiosqlite.Connection,
1535         user_id: int,
1536         size_bytes: int,
1537         duration_s: int,
1538         height: int,
1539         width: int,
1540     ) -> VideoRecord | None:
1541         """Tier-2 content dedup: query metadata COLUMNS, not the ``dedup_key``.
1542
1543         ADR 0011: ``dedup_key`` is a storage discriminator (``tg:doc:X`` vs
1544         ``tg:fp:...`` vs ``tg:msg:...``), NOT the read path. Items with a
1545         ``document_id`` are stored under ``tg:doc:X`` but their metadata columns
1546         are still populated ? querying the columns catches cross-key matches
1547         that a key-based lookup would miss (the Tier-2 regression: a re-upload
1548         with a new doc id + matching fingerprint must still be caught).
1549
1550         .. versionchanged:: Track B (review fix)
1551             Reverted from key-based lookup to column-based lookup, matching the v1
1552             pre-ADR-0011 behaviour that was regressed when :func:`compute_dedup_key`
1553             made the three tiers mutually-exclusive keys.
1554         """
1555         conn.row_factory = aiosqlite.Row
1556         async with conn.execute(
1557             """SELECT ci.*, s.provider_source_id AS src_pid
1558                FROM content_items ci
1559                JOIN sources s ON s.id = ci.source_id
1560                WHERE ci.account_id = ?
1561                AND ci.size_bytes = ?
```

```

1562         AND ci.duration_s = ?
1563         AND ci.height = ?
1564         AND ci.width = ?
1565     LIMIT 1""",
1566     (user_id, size_bytes, duration_s, height, width),
1567 ) as cur:
1568     row = await cur.fetchone()
1569 if row is None:
1570     return None
1571 return VideoRecord(
1572     id=row["id"],
1573     user_id=user_id,
1574     channel_id=_safe_int_cast(row["src_pid"], "provider_source_id"),
1575     message_id=_safe_int_cast(row["provider_message_id"], "provider_message_id"),
1576     text=row["text"],
1577     document_id=(
1578         _safe_int_cast(row["document_id"], "document_id")
1579         if row["document_id"] is not None
1580         else None
1581     ),
1582     duration_s=row["duration_s"],
1583     width=row["width"],
1584     height=row["height"],
1585     size_bytes=row["size_bytes"],
1586     mime_type=row["mime_type"],
1587     matched=bool(row["matched"]),
1588     forwarded=False,
1589     seen_at=row["seen_at"],
1590 )
1591
1592
1593 async def get_source_by_id(conn: aioredis.Connection, source_id: int) -> Source | None:
1594     """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1595     conn.row_factory = aioredis.Row
1596     async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1597         row = await cur.fetchone()
1598     return _row_to_source(row) if row else None
1599
1600
1601 async def set_source_watched(
1602     conn: aioredis.Connection, account_id: int, source_id: int, watched: bool

```

12. assistant (2026-07-06T21:26:21.171Z)

`get\_video` and `get\_video\_by\_document\_id` both route through `\_content\_item\_to\_video`, which now propagates `text` (line 1735 in the diff). So those read paths are fine.

Now let me examine the `source` variable flow in `on\_new\_message` carefully. There's a subtle issue: `source` is fetched via `get\_source\_by\_provider\_id`, then re-fetched inside the try. Let me read the full handler and check what happens on the exclude-veto path ? specifically whether a topic-excluded video still gets processed correctly.

13. user (2026-07-06T21:26:22.608Z)

```

230         ):
231             existing = await db.get_video_by_content_fingerprint(
232                 conn,
233                 user.id,
234                 meta.size_bytes,
235                 meta.duration_s,
236                 meta.height,
237                 meta.width,
238             )
239             if existing is not None:
240                 log.info(

```

```

241         "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
242         "channel=%s msg=%s original_channel=%s original_msg=%s",
243         user.id,
244         meta.size_bytes,
245         meta.duration_s,
246         meta.height,
247         meta.width,
248         channel_id,
249         message_id,
250         existing.channel_id,
251         existing.message_id,
252     )
253     return
254
255     source = await db.get_source_by_provider_id(conn, user.id, "telegram",
str(channel_id))
256
257     # Lazily record the channel (best-effort; failures are non-fatal).
258     try:
259         channel = await db.upsert_channel(
260             conn,
261             Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
262         )
263         if channel.id is not None:
264             source = await db.get_source_by_id(conn, channel.id)
265     except Exception:
266         log.warning(
267             "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
268         )
269
270     text = _message_text(message)
271     result = engine_holder[ENGINE_KEY].evaluate(meta)
272     matched = result.matched
273     reasons = list(result.reasons)
274     if matched and source is not None:
275         topic_result = await _evaluate_topic_preferences(
276             conn,
277             user.id,
278             source,
279             _content_item_preview(
280                 user.id,
281                 source,
282                 message_id,
283                 document_id,
284                 meta,
285                 text,
286                 matched,
287             ),
288         )
289         if topic_result is not None:
290             reasons.append(topic_result.reason)
291             matched = topic_result.passed
292
293     record = VideoRecord(
294         user_id=user.id,
295         channel_id=channel_id,
296         message_id=message_id,
297         text=text,
298         document_id=document_id,
299         duration_s=meta.duration_s,
300         width=meta.width,
301         height=meta.height,
302         size_bytes=meta.size_bytes,

```

```

303         mime_type=meta.mime_type,
304         matched=matched,
305     )
306     try:
307         inserted = await db.insert_video(conn, record)
308     except sqlite3.IntegrityError:
309         # Lost a dedup race: a concurrent delivery of the same file
310         # (same document_id or same message instance) won the insert.
311         # The three-tier check above already classified this as new, so
312         # the collision means another task is processing (or has
313         # processed) it ? log and skip.
314         log.info(
315             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
316             user.id,
317             channel_id,
318             message_id,
319             document_id,
320         )
321         return
322     log.info(
323         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
324         user.id,
325         channel_id,
326         message_id,
327         matched,
328         reasons,
329     )
330
331     if matched:
332         # Build link text for LINK/BOTH modes (None for FORWARD).
333         link_text = None
334         if delivery_mode != DeliveryMode.FORWARD:
335             channel_title = _channel_title(event)
336             link_text = build_link_text(
337                 message_link=getattr(message, "link", None),
338                 duration_s=meta.duration_s,
339                 height=meta.height,
340                 size_bytes=meta.size_bytes,
341                 channel_title=channel_title,
342             )
343         await deliver(
344             client,
345             message,
346             conn,
347             inserted.id,
348             delivery_mode,
349             link_text=link_text,
350             target=delivery_target,
351         )
352
353     return on_new_message
354
355     # --- helpers -----
356
357
358
359     def _extract_document_id(message: Message) -> int | None:

```

14. assistant (2026-07-06T21:26:27.272Z)

Now I have the key picture. Let me look at the `get\_source\_by\_provider\_id` signature ? the `on\_new\_message` calls it with positional args `(conn, user.id, "telegram", str(channel\_id))`. Let me verify the signature, and check `get\_default\_policy` for what `account\_id` means (user.id vs account.id).

15. user (2026-07-06T21:26:28.472Z)

```
1155     async def get_source_by_provider_id(
1156         conn: aiomysql.Connection, account_id: int, provider: str, provider_source_id: str
1157     ) -> Source | None:
1158         """Resolve a source by its provider-native id."""
1159         conn.row_factory = aiomysql.Row
1160         async with conn.execute(
1161             """SELECT * FROM sources
1162                WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1163             (account_id, provider, provider_source_id),
1164         ) as cur:
1165             row = await cur.fetchone()
1166             return _row_to_source(row) if row else None
1167
1168
1169     # --- ContentItem repos (ADR 0011) -----
1170
1171
1172     async def insert_content_item(conn: aiomysql.Connection, item: ContentItem) ->
ContentItem:
1173         """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1174
1175         The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1176         unique constraints guard against duplicates:
1177         - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1178         - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1179         """
```

16. assistant (2026-07-06T21:26:36.346Z)

Now, a key concern: the `on_new_message` computes `source` via `get_source_by_provider_id` AND via `get_source_by_id(channel.id)`. Note `upsert_channel` returns a `Channel` whose `.id` ? is that the source id? Let me check `upsert_channel` and how `channel.id` relates to `source.id`. This matters because `_content_item_preview` asserts `source.id is not None` and uses `source.provider` / `source.provider_source_id`.

Let me focus on the more impactful concern for delivery correctness. Let me check the `exclude_veto` path and whether `matched=False` from topics still correctly gates. That looks correct in the code (`matched` updated, `gates` insert's `matched` flag and `delivery`).

Let me now run a repro to confirm the wiring works and check the `dedup_key` computed in `preview` vs what `insert_video` computes ? if they differ, is there a correctness impact? Let me check `insert_video`'s `dedup` computation.

17. user (2026-07-06T21:26:36.969Z)

```
1660         if absent), computes the provider-neutral ``dedup_key`` via
1661         :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1662         ``VideoRecord`` shape is reconstructed from the inserted row.
1663         """
1664         src = await upsert_source(
1665             conn,
1666             Source(
1667                 account_id=video.user_id,
1668                 provider="telegram",
1669                 provider_source_id=str(video.channel_id),
1670                 kind="channel",
1671                 watched=False,
1672             ),
1673         )
1674         assert src.id is not None # upsert_source populates the surrogate PK
1675         item = ContentItem(
1676             account_id=video.user_id,
1677             source_id=src.id,
```

```

1678         provider="telegram",
1679         provider_message_id=str(video.message_id),
1680         content_type="video",
1681         text=video.text,
1682         document_id=str(video.document_id) if video.document_id is not None else None,
1683         duration_s=video.duration_s,
1684         width=video.width,
1685         height=video.height,
1686         size_bytes=video.size_bytes,
1687         mime_type=video.mime_type,
1688         dedup_key=compute_dedup_key(
1689             provider="telegram",
1690             document_id=(str(video.document_id) if video.document_id is not None else
None),
1691             source_id=src.id,
1692             provider_message_id=str(video.message_id),
1693             size_bytes=video.size_bytes,
1694             duration_s=video.duration_s,
1695             height=video.height,
1696             width=video.width,
1697         ),
1698         matched=video.matched,
1699         seen_at=video.seen_at,
1700     )
1701     item = await insert_content_item(conn, item)
1702     return _content_item_to_video(item, src, video.user_id, video.channel_id)
1703
1704
1705 async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
1706     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1707
1708     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1709     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1710     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1711     delivery still targets the single ``delivery_target`` and retried delivery
1712     is harmless (forwards are idempotent). This shim is retained so existing
1713     call sites in ``delivery.py`` keep working without a flag day.
1714     """
1715     # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1716     # The row's ``matched`` flag (set at insert time) is the surviving signal.
1717     return None
1718
1719

```

18. assistant (2026-07-06T21:26:44.857Z)

The `dedup_key` computed in `_content_item_preview` is indeed throwaway (the real one is computed in `insert_video` from the source that `upsert_source` returns). Confirmed wasteful but no correctness impact since the preview `ContentItem` is only fed to `TopicPreferenceStage` which only reads `source.title` and `content_item.text`.

Now let me look at the tests added to understand coverage, and confirm which new wiring lines are uncovered. Let me check the models: does `text` in `VideoRecord` get an index constraint issue? No. Let me now run a repro of the wiring path to verify it works end-to-end and check for a real bug.

Let me check one important thing: `_message_text` reads `message.message`. But for Telethon, `message.message` on a caption-less video could be empty string. That's handled. But there's a subtler concern ? does `get_video_by_content_fingerprint` SELECT include the `text` column? It uses `ci.*` so yes. Good.

Let me check the test files to see what wiring is covered.

19. user (2026-07-06T21:26:47.026Z)

```

diff --git a/tests/test_db.py b/tests/test_db.py
index 8692285..54e14bb 100644
--- a/tests/test_db.py
+++ b/tests/test_db.py
@@ -230,6 +230,7 @@ async def test_insert_and_get_video_round_trip(conn: aioredis.Connection)
-> No
        user_id=USER_ID,
        channel_id=CHANNEL_ID,
        message_id=42,
+    text="AI philosophy caption",
        duration_s=120,
        width=1920,
        height=1080,
@@ -244,12 +245,19 @@ async def test_insert_and_get_video_round_trip(conn: aioredis.Connection)
-> No
        fetched = await db.get_video(conn, USER_ID, CHANNEL_ID, 42)
        assert fetched is not None
        assert fetched.duration_s == 120
+    assert fetched.text == "AI philosophy caption"
        assert fetched.height == 1080
        assert fetched.size_bytes == 5_000_000
        assert fetched.mime_type == "video/mp4"
        assert fetched.matched is True
        assert fetched.forwarded is False

+    src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", str(CHANNEL_ID))
+    assert src is not None and src.id is not None
+    item = await db.get_content_item_by_message(conn, USER_ID, src.id, "42")
+    assert item is not None
+    assert item.text == "AI philosophy caption"
+
    async def test_insert_video_dedup_collision(conn: aioredis.Connection) -> None:
        await _seed_user(conn)
@@ -470,6 +478,59 @@ async def test_save_filter_config_overwrites(conn: aioredis.Connection) ->
None
        assert loaded.max_duration_s == 200

+async def test_save_filter_config_preserves_unknown_policy_keys(
+    conn: aioredis.Connection,
+) -> None:
+    """ADR 0012: legacy config updates must not erase pipeline config keys."""
+    await _seed_user(conn)
+    await db.save_policy_config(
+        conn,
+        USER_ID,
+        {
+            "min_duration_s": 5,
+            "max_duration_s": 7200,
+            "topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"},
+            "security_checks": {"blocked_domains": ["sketchy.example"]},
+            "deep_inspect": True,
+        },
+    )
+    await db.save_filter_config(conn, USER_ID, FilterConfig(max_size_mb=512))
+
+    raw = await db.load_policy_config(conn, USER_ID)
+    assert raw["max_size_mb"] == 512
+    assert raw["topics"] == {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}
+    assert raw["security_checks"] == {"blocked_domains": ["sketchy.example"]}
+    assert raw["deep_inspect"] is True
+    loaded = await db.load_filter_config(conn, USER_ID)
+    assert loaded.max_size_mb == 512

```

```

+
+
+async def test_save_filter_config_preserves_existing_default_policy_name(
+    conn: aiomysqlite.Connection,
+) -> None:
+    """A default policy does not have to be named literally ``default``."""
+    await _seed_user(conn)
+    await db.upsert_policy(
+        conn,
+        Policy(
+            account_id=USER_ID,
+            name="topic-heavy",
+            config_json='{"topics":{"include":["AI"],"mode":"any"}}',
+            is_default=True,
+        ),
+    )
+
+    await db.save_filter_config(conn, USER_ID, FilterConfig(max_size_mb=256))
+
+    policy = await db.get_default_policy(conn, USER_ID)
+    assert policy is not None
+    assert policy.name == "topic-heavy"
+    raw = await db.load_policy_config(conn, USER_ID)
+    assert raw["topics"] == {"include": ["AI"], "mode": "any"}
+    assert raw["max_size_mb"] == 256
+
+
+async def test_filter_config_with_null_bounds_round_trips(conn: aiomysqlite.Connection) -> None:
+    """None bounds (rule disabled) must survive the JSON round trip."""
+    await _seed_user(conn)
diff --git a/tests/test_telethon_extraction.py b/tests/test_telethon_extraction.py
index 49e8cfd..3c5ab20 100644
--- a/tests/test_telethon_extraction.py
+++ b/tests/test_telethon_extraction.py
@@ -23,7 +23,12 @@ from telethon.tl.types import (
     PeerUser,
 )

-from tg_video_filter.telethon_client import _extract_document_id, _peer_id, extract_video_meta
+from tg_video_filter.telethon_client import (
+    _extract_document_id,
+    _message_text,
+    _peer_id,
+    extract_video_meta,
+)

CHANNEL_ID = 1001234567890

@@ -120,6 +125,25 @@ def test_text_message_returns_none() -> None:
    assert extract_video_meta(_make_text_message()) is None

+def test_message_text_strips_caption() -> None:
+    msg = _make_video_message()
+    msg.message = " AI philosophy clip "
+    assert _message_text(msg) == "AI philosophy clip"
+
+
+def test_message_text_empty_returns_none() -> None:
+    msg = _make_video_message()
+    msg.message = " "
+    assert _message_text(msg) is None
+
+
+def test_message_text_raw_text_fallback() -> None:

```

```

+   class Msg:
+       raw_text = "Bangalore explorations"
+
+   assert _message_text(Msg()) == "Bangalore explorations"
+
+
+   def test_photo_message_returns_none() -> None:
+       msg = _make_text_message()
+       msg.media = MessageMediaPhoto(photo=None) # not a Document

```

20. assistant (2026-07-06T21:26:51.747Z)

Now I've confirmed: the unit tests cover `\_message\_text`, `\_TopicPreferenceStage`, and the db shims ? but NOT `\_content\_item\_preview`, `\_evaluate\_topic\_preferences`, or the gating logic in `on_new_message/backfill`. Those are behind `pragma:no-cover`.

Let me now run a repro that exercises the actual wiring ? `\_evaluate\_topic\_preferences` + `\_content\_item\_preview` ? since this is the untested path and where a wiring bug would hide. Let me check `get_default_policy` semantics and test the veto path.

21. user (2026-07-06T21:26:59.666Z)

```

['get_account', 'get_account_by_telegram_id', 'get_user', 'list_account_sources',
'list_enabled_users', 'list_videos_for_user', 'upsert_account', 'upsert_user']

```

22. assistant (2026-07-06T21:27:02.778Z)

Let me look at how `test_db` seeds a user to mirror it, then build the wiring repro.

23. user (2026-07-06T21:27:04.761Z)

```

tests/test_db.py:26:USER_ID = 111
tests/test_db.py:27:CHANNEL_ID = -1001234567890
tests/test_db.py:39:async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) ->
User:
tests/test_db.py:75:     created = await _seed_user(conn)
tests/test_db.py:89:     await _seed_user(conn)
tests/test_db.py:205:     await _seed_user(conn)
tests/test_db.py:228:     await _seed_user(conn)
tests/test_db.py:263:     await _seed_user(conn)
tests/test_db.py:280:     await _seed_user(conn)
tests/test_db.py:295:     await _seed_user(conn)
tests/test_db.py:341:     await _seed_user(conn)
tests/test_db.py:358:     await _seed_user(conn)
tests/test_db.py:364:     await _seed_user(conn)
tests/test_db.py:389:     await _seed_user(conn)
tests/test_db.py:413:     await _seed_user(conn)
tests/test_db.py:439:     await _seed_user(conn)
tests/test_db.py:458:     await _seed_user(conn)
tests/test_db.py:464:     await _seed_user(conn)
tests/test_db.py:474:     await _seed_user(conn)
tests/test_db.py:485:     await _seed_user(conn)

```

24. user (2026-07-06T21:27:07.012Z)

```

39     async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
40         return await db.upsert_user(conn, User(id=uid, phone="+15555550100"))
41
42
43     # --- schema / init -----
44
45
46     async def test_init_db_creates_tables(tmp_path: object) -> None:
47         db_path = str(tmp_path) + "/test.db" # type: ignore[operator]

```

```

48         await db.init_db(db_path)
49         # Re-open and confirm all expected tables exist. Post-ADR-0011 the
50         # provider-neutral tables are sources/content_items/policies (plus the
51         # empty Track-C tables destinations/routing_rules/delivery_attempts);
52         # the legacy channels/videos/filter_config are absent on fresh DBs
53         # (they only exist as *_deprecated on migrated legacy DBs).
54         async with (
55             aiosqlite.connect(db_path) as c,
56             c.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name") as
cur,
57         ):
58             names = [r[0] for r in await cur.fetchall()]
59             assert set(names) >= {
60                 "accounts",
61                 "telegram_sessions",
62                 "sources",
63                 "content_items",
64                 "policies",
65                 "destinations",
66                 "routing_rules",
67                 "delivery_attempts",
68             }
69
70
71         # --- user repos -----
72
73
74         async def test_upsert_and_get_user(conn: aiosqlite.Connection) -> None:

```

25. assistant (2026-07-06T21:27:12.145Z)

Now let me write a comprehensive wiring repro that exercises the exact untested path: seed user + source + default policy with topics, then run `\_evaluate\_topic\_preferences` with the `\_content\_item\_preview` builder, testing include-match, exclude-veto, and no-policy.

26. user (2026-07-06T21:27:27.101Z)

```

source: 1 AI Philosophy Daily telegram -1001234567890
A include via title: True matched include topic: AI
B exclude veto: False excluded topic matched: Crypto
C no policy: None

```

27. assistant (2026-07-06T21:27:32.312Z)

Wiring works as designed. Now let me investigate a subtle but important thing: the default policy `account\_id`. In `on\_new\_message`, `\_evaluate\_topic\_preferences(conn, user.id, ...)` uses `user.id` as `account\_id` for `get\_default\_policy`. Is `user.id` == `account\_id` in this schema? In `test\_db`, `save\_policy\_config(conn, USER\_ID, ...)` and `get\_default\_policy(conn, account\_id)` both use `USER\_ID`, and `\_seed\_user` creates a User with that id. Let me confirm accounts vs users relationship ? whether policies key on account id which may differ from user id.

28. user (2026-07-06T21:27:34.274Z)

```

src/tg_video_filter/db.py:145:ON policies(account_id) WHERE is_default = 1;
src/tg_video_filter/db.py:626:         INSERT OR IGNORE INTO policies (account_id, name,
config_json, is_default,
src/tg_video_filter/db.py:640:         "ON policies(account_id) WHERE is_default = 1"
src/tg_video_filter/db.py:726:         JOIN policies p ON p.account_id = s.account_id AND
p.is_default = 1
src/tg_video_filter/db.py:1257:         """INSERT INTO policies (account_id, name, config_json,
is_default,
src/tg_video_filter/db.py:1276:         "SELECT * FROM policies WHERE account_id = ? AND name =
?",
src/tg_video_filter/db.py:1288:         "SELECT * FROM policies WHERE account_id = ? AND

```

```
is_default = 1",
src/tg_video_filter/db.py:1606:      "UPDATE sources SET watched = ? WHERE id = ? AND
account_id = ?",
src/tg_video_filter/db.py:1617:      query = "SELECT * FROM sources WHERE account_id = ?"
```

29. assistant (2026-07-06T21:27:38.174Z)

The existing code consistently uses `user.id` as `account\_id` throughout (e.g. `load\_filter\_config(conn, user\_id)` calls `get\_default\_policy(conn, user\_id)`). So F2's `\_evaluate\_topic\_preferences(conn, user.id, ...)` is consistent with the established convention. Not a bug.

Now let me verify the coverage claim ? which specific new lines are uncovered. Let me run the actual test suite with coverage on the two source files to confirm the untested wiring.

30. user (2026-07-06T21:27:52.249Z)

```
..... [ 14%]
..... [ 28%]
..... [ 42%]
..... [ 56%]
..... [ 70%]
..... [ 84%]
..... [ 98%]
..... [100%]
===== tests coverage =====
_____ coverage: platform win32, python 3.13.13-final-0 _____
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src\tg_video_filter__init__.py	1	0	0	0	100%	
src\tg_video_filter__main__.py	7	0	0	0	100%	
src\tg_video_filter\backfill.py	49	1	16	1	97%	88
src\tg_video_filter\bootstrap.py	23	0	0	0	100%	
src\tg_video_filter\command_service.py	218	8	94	3	96%	226-227,
242-243, 262-263, 369, 424, 429->428						
src\tg_video_filter\commands.py	46	0	12	0	100%	
src\tg_video_filter\config.py	26	1	2	1	93%	64
src\tg_video_filter\db.py	352	17	56	8	94%	524, 653-656,
687, 690, 910, 1304, 1322, 1474-1475, 1487, 1529, 1783-1791, 1887						
src\tg_video_filter\delivery.py	102	1	48	3	97%	147, 188->190,
256->258						
src\tg_video_filter\filters__init__.py	39	0	18	0	100%	
src\tg_video_filter\filters\base.py	26	0	0	0	100%	
src\tg_video_filter\filters\content_type.py	23	0	12	0	100%	
src\tg_video_filter\filters\duration.py	15	0	6	0	100%	
src\tg_video_filter\filters\resolution.py	13	0	6	0	100%	
src\tg_video_filter\filters\size.py	18	0	6	0	100%	
src\tg_video_filter\manager.py	36	0	8	0	100%	
src\tg_video_filter\models.py	163	1	8	1	99%	364
src\tg_video_filter\policy.py	101	6	34	6	91%	61, 83, 98,
110, 112->108, 140, 150						
src\tg_video_filter\telethon_client.py	79	10	32	5	85%	78, 95->108,
353, 368, 380, 395-396, 430-433, 463->exit						
TOTAL	1337	45	358	28	96%	

Required test coverage of 80.0% reached. Total coverage: 95.58%
513 passed in 8.82s

31. assistant (2026-07-06T21:27:55.809Z)

Now let me identify what those uncovered lines in telethon_client.py (395-396, 430-433) and policy.py are.

32. user (2026-07-06T21:27:57.081Z)

```
388         message_id: int,
389         document_id: int | None,
390         meta: VideoMeta,
391         text: str | None,
392         matched: bool,
393     ) -> ContentItem:
394         """Build the ContentItem shape used by policy stages before insert."""
395         assert source.id is not None
396         return ContentItem(
397             account_id=account_id,
398             source_id=source.id,
399             provider=source.provider,
400             provider_message_id=str(message_id),
401             content_type="video",
402             text=text,
403             document_id=str(document_id) if document_id is not None else None,
404             duration_s=meta.duration_s,
405             width=meta.width,
406             height=meta.height,
407             size_bytes=meta.size_bytes,
408             mime_type=meta.mime_type,
409             dedup_key=db.compute_dedup_key(
410                 provider=source.provider,
411                 document_id=str(document_id) if document_id is not None else None,
412                 source_id=source.id,
413                 provider_message_id=str(message_id),
414                 size_bytes=meta.size_bytes,
415                 duration_s=meta.duration_s,
416                 height=meta.height,
417                 width=meta.width,
418             ),
419             matched=matched,
420         )
421
422
423     async def _evaluate_topic_preferences(
424         conn: aiosqlite.Connection,
425         account_id: int,
426         source: Source,
427         content_item: ContentItem,
428     ) -> StageResult | None:
429         """Evaluate ADR 0012 topic preferences for a candidate item."""
430         policy = await db.get_default_policy(conn, account_id)
431         if policy is None:
432             return None
433         return await TopicPreferenceStage().evaluate(
434             PolicyContext(content_item=content_item, source=source, policy=policy)
435         )
436
437
```

33. assistant (2026-07-06T21:28:08.032Z)

Confirmed: `\_content\_item\_preview` (395-420) and `\_evaluate\_topic\_preferences` (430-433) are entirely uncovered by the suite ? even though they are NOT behind pragma:no-cover (they're plain helpers). The coverage shows lines 395-396 and 430-433 missed. My repro just exercised them successfully, proving a test is trivially possible. This is the "happy path missed" the task asks me to flag.

Now let me investigate one more potential real bug. Look at the policy.py
`load\_topic\_preferences` ? it does `json.loads(policy.config\_json)`. But
`load\_filter\_config`/`save\_filter\_config` now store config with `model\_dump(mode="json")`
merged. Let me verify a full round trip: a `/set` command (F1) that calls `save\_filter\_config` ?

does the resulting config_json still parse in TopicPreferenceStage? The task says the main reviewer verified this. Let me instead check a different edge: what happens to `matched` reasons when topic result is appended but `source is None` (unwatched/failed upsert). Then topics are silently skipped and video delivered on filter match alone ? is that intended? Yes, consistent with "dormant until config path exists."

Let me check the `all` mode edge case in the stage and a potential bug: in `\_matched\_topics`, includes are matched by casefolded substring, and `missing` compares `topic not in matched\_includes`. But `matched\_includes` contains the ORIGINAL-cased topics from `prefs.include`. Let me verify there's no case mismatch bug in the `all` mode.

34. user (2026-07-06T21:28:19.030Z)

all-mode both present (case-insensitive): True matched all include topics
all-mode one missing: False missing required topic: Robotics
invalid mode: False invalid topic config: topics.mode must be 'any' or 'all'

35. assistant (2026-07-06T21:28:23.466Z)

All-mode works correctly. Note the invalid-config path fails-closed (returns `passed=False`), which means a malformed `topics` config causes a matched video to be DROPPED. Since `load\_filter\_config` raises on malformed config elsewhere, and topics can only be set via `save\_policy\_config` (no user command), this is a latent design note but low severity.

Now let me confirm the important design-consistency finding about `evaluate\_pipeline` being unused. Let me verify it's not called anywhere in production.

36. user (2026-07-06T21:28:24.405Z)

```
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\policy.py:176:async def evaluate_pipeline(  
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\policy.py:196:     "evaluate_pipeline",
```

37. assistant (2026-07-06T21:28:31.800Z)

Confirmed: `evaluate\_pipeline` is defined but never called in `src/`. F2 hand-wires `TopicPreferenceStage().evaluate()` directly via `\_evaluate\_topic\_preferences`, bypassing the orchestrator. This is a design-consistency note (F1's orchestrator stays unused in production).

Now, one more real-bug check. In `on\_new\_message`, `reasons.append(topic\_result.reason)` appends the topic reason, but this is only for the log line. Fine. But notice: on the exclude-veto path (matched flipped to False), the video is still INSERTED with `matched=False` and text stored ? correct. And delivery is skipped. Good.

Let me check one potential completeness gap: does `\_message\_text` handle the case where Telethon's `message.message` is `None` but `raw\_text` exists AND is non-string? Covered by isinstance check. Fine.

Let me verify the backfill's `document\_id` type. In backfill, `document\_id` is passed to `\_content\_item\_preview` with `# type: ignore[arg-type]`, suggesting document_id may not be `int` | `None` there. Let me check.

38. user (2026-07-06T21:28:32.402Z)

```
200             conn,  
201             user.id,  
202             meta.size_bytes,  
203             meta.duration_s,  
204             meta.height,  
205             meta.width,  
206         )  
207         if existing is not None:  
208             log.info(  
209                 "backfill_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s  
210                 "
```

```

210         "channel=%s msg=%s original_channel=%s original_msg=%s",
211         user.id,
212         meta.size_bytes,
213         meta.duration_s,
214         meta.height,
215         meta.width,
216         channel_id,
217         message_id,
218         existing.channel_id,
219         existing.message_id,
220     )
221     continue
222
223     summary["videos_scanned"] += 1
224     dialog_videos += 1
225
226     source = None
227     try:
228         channel = await db.upsert_channel(
229             conn,
230             Channel(user_id=user.id, channel_id=channel_id, title=title),
231         )
232         if channel.id is not None:
233             source = await db.get_source_by_id(conn, channel.id)
234     except Exception:
235         log.warning(
236             "failed to upsert channel %s for user %s during backfill",
237             channel_id,
238             user.id,
239             exc_info=True,
240         )
241
242     text = _message_text(message)
243     result = engine.evaluate(meta)
244     matched = result.matched
245     if matched and source is not None:
246         topic_result = await _evaluate_topic_preferences(
247             conn,
248             user.id,
249             source,
250             _content_item_preview(
251                 user.id,
252                 source,
253                 message_id,
254                 document_id, # type: ignore[arg-type]
255                 meta,
256                 text,
257                 matched,
258             ),
259         )
260         if topic_result is not None:
261             matched = topic_result.passed
262
263     record = VideoRecord(
264         user_id=user.id,
265         channel_id=channel_id,
266         message_id=message_id,
267         text=text,
268         document_id=document_id, # type: ignore[arg-type]
269         duration_s=meta.duration_s,

```

39. assistant (2026-07-06T21:28:39.346Z)

Now I have a complete picture. Both handlers store `text` and gate on topic results consistently. The backfill also stores text (answering that sub-question: yes).

One notable behavioral inconsistency: in `on\_new\_message`, the topic `reason` is appended to `reasons` and logged (`video\_seen ... reasons=%s`). In `backfill\_user`, the topic reason is NOT captured into any reasons/log ? `matched = topic\_result.passed` but the reason is discarded. Minor observability inconsistency between the two wired paths. Low severity.

Let me verify the backfill log line to confirm reasons aren't logged there.

40. user (2026-07-06T21:28:40.065Z)

```
269             duration_s=meta.duration_s,
270             width=meta.width,
271             height=meta.height,
272             size_bytes=meta.size_bytes,
273             mime_type=meta.mime_type,
274             matched=matched,
275         )
276         inserted = await db.insert_video(conn, record)
277
278         if matched:
279             summary["videos_matched"] += 1
280             dialog_matched += 1
281             link_text = build_link_text(
282                 message_link=getattr(message, "link", None),
283                 duration_s=meta.duration_s,
284                 height=meta.height,
285                 size_bytes=meta.size_bytes,
286                 channel_title=title, # from the dialog object
287             )
288             await deliver(
289                 client,
290                 message,
291                 conn,
292                 inserted.id or 0,
293                 delivery_mode,
294                 link_text=link_text,
295                 target=delivery_target,
296             )
297
298             # Periodic progress (at most once per 30 seconds per dialog).
299             now = asyncio.get_event_loop().time()
300             if progress_callback and now - last_progress > 30:
301                 await progress_callback(
302                     f"    #{title}: {dialog_total} msgs, {dialog_videos} videos,
303 {dialog_matched} matched..."
304                 )
305                 last_progress = now
306
307         except FloodWaitError as e:
308             wait = e.seconds + 1 # small extra buffer
309             log.warning("backfill_flood_wait channel=%s seconds=%s", title, e.seconds)
```

41. assistant (2026-07-06T21:28:44.142Z)

Confirmed ? backfill discards the topic reason. Now I have all findings. Let me also quickly confirm there's no test at all for `\_content\_item\_preview`/`\_evaluate\_topic\_preferences` in the test suite (to be certain the happy-path is missed).

42. user (2026-07-06T21:28:45.322Z)

No files found

43. assistant (2026-07-06T21:28:52.159Z)

Confirmed ? zero test references. The two helpers are exported in `\_\_all\_\_` but never tested.

Let me do a final check on one potential correctness concern: `\_content\_item\_preview` builds a ContentItem with `provider=source.provider`, but `compute\_dedup\_key` is called with `provider=source.provider`. If `source.provider` were ever something other than "telegram", is that consistent? It's throwaway anyway. No impact.

I have my findings. Let me compile the structured report.

44. user (2026-07-06T21:29:35.042Z)

Structured output provided successfully